

CS285 LECTURE 8 • ZERO-TO-HERO FIELD MANUAL

Continuous Q-Learning, TD3, & Soft Actor-Critic (SAC)

From Scratch to Mastery: Conquering Continuous Robot Action Spaces, Overcoming the Deadly Triad, and Unleashing Entropy-Driven Curiosity

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab)

Scope: Continuous $\max Q$, Clipped Double-Q, Target Networks, Reparameterization, & MaxEnt RL

1. Why Classic Q-Learning Fails on Physical Robots

In 2015, DeepMind's **Deep Q-Networks (DQN)** astonished computer science by learning to play 49 Atari 2600 video games directly from raw pixels, achieving superhuman performance on games like *Breakout*, *Pong*, and *Space Invaders*. Naturally, robotics researchers immediately attempted to apply DQN directly to physical robotic arms, quadrupeds, and autonomous vehicles. **Every single attempt failed catastrophically.**

To understand why DQN conquered Atari but failed on physical hardware, we must examine the single mathematical operation at the absolute core of Q-learning:

$$a^*(s) = \arg \max_{a \in \mathcal{A}} Q(s, a) \quad \text{and} \quad y = r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q(s', a')$$



EVERYDAY ANALOGY: THE RESTAURANT MENU VS THE FOGGY MOUNTAIN BEACH

Think of the difference between discrete action spaces and continuous action spaces:

- **Atari Video Game (The 4-Item Menu):** At any moment in *Pong*, you can only press 4 buttons: [UP, DOWN, LEFT, RIGHT]. Evaluating $\max_a Q(s, a)$ is identical to reading a 4-item dinner menu. The neural network computes 4 scalar numbers: [2.1, 4.5, 9.8, 1.2]. Your computer runs a trivial `torch.argmax()` in 0.0001 milliseconds and picks button 3.
- **Robotic Arm / Dual-Arm Manipulation (The Foggy Mountain Beach):** In our dual-arm slicing robot, an action is not a button click. It is a continuous 14-dimensional vector of real numbers specifying target joint motor torques:

$$\mathbf{a} = [\tau_1, \tau_2, \dots, \tau_{14}] \in [-10.0, +10.0]^{14} \text{ Nm}$$

There are no 4 choices. There are **uncountably infinite choices**! Evaluating $\max_a Q(s, a)$ is like being dropped blindfolded onto a massive, foggy mountain beach with endless rolling sand dunes, and being commanded to find the *single highest grain of sand* across the entire mountain range in under **1 millisecond**.

ATARI: 4 DISCRETE CHOICES

Up: 4.2 Down: 2.1 **Right: 9.8 (MAX!)**
Instantaneous

`torch.argmax()` over 4 scalars vs. PHYSICAL ROBOT: 14-DoF CONTINUOUS SURFACE Optimal Continuous Peak a^* An Actor network $\pi_\theta(s)$ is trained to climb directly to the peak!

1.1 Why Can't We Just Discretize the Robot's Actions?

A naive engineer might say: "Why not discretize each motor into 10 bins (e.g. -10, -8, -6, ..., +8, +10 Nm) and use standard DQN?"

Let's do the simple combinatorics. If a robot has d joints and we divide each joint into K bins, the total discrete action space size is:

$$|\mathcal{A}_{\text{discrete}}| = K^d$$

- **1 Joint** ($d = 1, K = 10$): $10^1 = 10$ choices. (Trivial).
- **7-DoF Single Arm** ($d = 7, K = 10$): $10^7 = 10,000,000$ choices. To pick one action, DQN must run 10 million forward passes through the Q-network every 10 milliseconds. Impossible!
- **14-DoF Dual-Arm Slicing System** ($d = 14, K = 10$): $10^{14} = 100,000,000,000,000$ choices (100 trillion discrete actions). Running $\arg \max$ across 100 trillion options would take a supercomputer several weeks for a single robot time-step!

Furthermore, coarse discretization destroys dexterous robotic skills: if a knife needs exactly 3.45 Nm of downward force to break a grape skin without crushing the pulp, a discretized choice between 2.0 Nm (fails to puncture) and 5.0 Nm (crushes into soup) means the robot can *never* perform the task successfully.

2. The Actor-Maximizer Solution: The Evolution to Soft Actor-Critic

To solve the continuous $\arg \max$ crisis without combinatorial explosion, the RL community developed the **Actor-Critic paradigm for continuous value learning** across three landmark algorithmic generations:

Algorithm	Authors & Year	Core Breakthrough Mechanism	Fatal Limitation / Why It Was Replaced
DDPG	Lillicrap et al. (DeepMind, 2015)	Deterministic Policy Gradient: Trains a neural network Actor $\mu_\theta(s)$ to directly output the continuous action that maximizes the Critic $Q_\phi(s, a)$. Uses chain rule: $\nabla_\theta Q(s, \mu_\theta(s)) = \nabla_a Q \nabla_\theta \mu$.	Severe Overestimation Bias & Hyperparameter Fragility: The Critic severely overestimates Q-values. If Q overestimates, the Actor chases hallucinated spikes and gets trapped in catastrophic policy collapse.
TD3 (Twin Delayed DDPG)	Fujimoto et al. (McGill, 2018)	Clipped Double-Q & Delayed Updates: Trains two independent critics (Q_1, Q_2), takes their minimum for Bellman targets, delays policy updates, and adds target noise.	Deterministic Policy Fragility: While stable, TD3 is a deterministic policy with additive Gaussian exploration noise ($\epsilon \sim \mathcal{N}(0, \sigma)$). It cannot learn multi-modal policies and easily gets stuck in sub-optimal local minima.
SAC (Soft Actor-Critic)	Haarnoja et al. (UC Berkeley, 2018)	Maximum Entropy RL + Twin Critics: Optimizes expected reward PLUS action entropy $\mathcal{H}(\pi)$. Uses the Reparameterization Trick and automatic temperature tuning.	The Gold Standard of Continuous Robotics: Unmatched sample efficiency, extreme robustness to hyperparameter changes, and continuous exploratory curiosity.

2.1 The Deterministic Policy Gradient (Chain Rule Parameter Anatomy)

How does the Actor actually maximize the Critic without an exhaustive search? In DDPG and TD3, we update the Actor parameters θ using the chain rule:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [\nabla_a Q_\phi(s, a) \Big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s)]$$

Mathematical Term	Formal Identity	Physical & Intuitive Meaning	Tensor Shape
$\nabla_a Q_\phi(s, a) \Big _{a=\mu_\theta(s)}$	Critic Action Gradient	The Critic's Uphill Compass: Tells the robot: "If you increase joint 3 torque by +0.1 Nm, your Q-value will rise by +4.2 points." The slope of the Q-surface with respect to motor torques.	[Batch, Act_Dim] (e.g. [256, 14])

Mathematical Term	Formal Identity	Physical & Intuitive Meaning	Tensor Shape
$\nabla_{\theta} \mu_{\theta}(s)$	Actor Parameter Jacobian	The Motor Control Levers: Tells the robot how its internal neural network weights θ must shift in order to change the output motor torque in that requested uphill direction.	[Batch, Act_Dim, Num_Weights]
$\nabla_{\theta} J(\theta)$	Policy Gradient Vector	The combined gradient vector that nudges the actor's weights so that its next predicted actions slide directly uphill toward the peak of the critic's Q-landscape!	[Num_Weights]



THE MOUNTAIN CLIMBER ANALOGY (CRITIC AS TOPOGRAPHER, ACTOR AS CLIMBER)

The Critic is an expert topographer looking down on the fog-covered mountain range, mapping where the high-altitude peaks and treacherous ravines lie. The Actor is a hiker standing on the mountain slope. Every time step, the Critic radios the hiker: *"Take two steps Northeast—that is the steepest uphill route!"* The hiker takes two steps Northeast. No random guessing or exhaustive search is required; the hiker follows the gradient directly uphill!

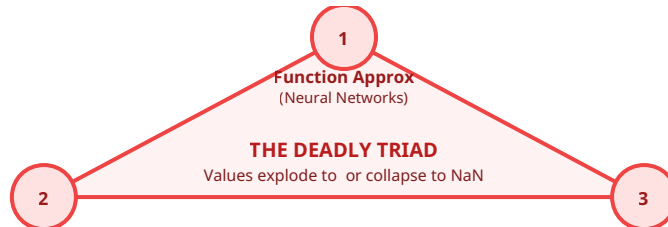
3. Four Real-World Industrial Case Studies: Continuous Action Spaces

Continuous control is not merely a theoretical construct; it is the universal language of physical robotics, aerospace, and real-world industrial infrastructure:

Application Domain	Continuous Action Vector $\mathbf{a}$	Continuous State Vector $\mathbf{s}$	Why Discretization Fails Completely
1. Dual-Arm Fruit Slicing (Robotics Thesis)	14 continuous joint motor torques: $\tau \in [-15.0, +15.0]$ Nm (7-DoF left arm holding fork + 7-DoF right arm holding knife).	28 joint positions & velocities +12 6-axis F/T wrench signals +3 knife tip coordinates +1 pulp deformation scalar.	If downward knife force discretizes to $[0, 5, 10]$ N, slicing a soft strawberry ($F_{\text{puncture}} \approx 2.3$ N) either skims the surface or pulverizes it into juice. Exact continuous compliance is mandatory.
2. Autonomous Highway Merging (Self-Driving Vehicles)	2 continuous controls: Steering wheel angle $\delta \in [-30^\circ, +30^\circ]$ and brake/throttle pressure $u \in [-1.0, +1.0]$.	64-dim vector: Ego vehicle velocity, acceleration, lane offset, plus relative (x, y, v_x, v_y) of the 10 nearest highway vehicles.	Discrete steering (e.g. $[-5^\circ, 0^\circ, +5^\circ]$) causes violent, jerky oscillations at 120 km/h, destabilizing vehicle chassis dynamics and triggering fatal rollovers.
3. Quadruped Locomotion (Unitree Go2 / ANYmal)	12 continuous joint target angles $\theta_{\text{target}} \in [-\pi, +\pi]$ fed to low-level 500 Hz PD joint controllers.	48-dim vector: Base orientation quaternion, angular velocity, linear velocity, 12 joint angles, 12 joint velocities, 4 foot contact flags.	Landing dynamic backflips requires smooth continuous ground reaction force absorption. Discretized joint steps shatter gearbox teeth upon dynamic impact.
4. Exothermic Chemical Reactor (Process Engineering)	3 continuous actuator setpoints: Coolant valve opening $\% \in [0.0, 100.0]$, reagent pump rate L/min, and agitator RPM.	16 sensor readings: Core reactor temperature, jacket temperature, internal pressure, chemical concentration, pH, viscosity.	Discretizing coolant flow by 10% increments causes temperature hunting cycles that cross thermal runaway thresholds, resulting in explosive reactor overpressure.

4. The Deadly Triad & Why Off-Policy Continuous RL Diverges

Richard Sutton proved in 1988 that if a reinforcement learning algorithm combines three specific elements, the value function parameters can mathematically diverge to $+\infty$ or $-\infty$:



Why does this deadly divergence happen?

- Function Approximation:** A deep neural network cannot memorize every state independently; changing the Q-value of state s_1 inadvertently shifts the Q-value of millions of nearby states $s_2, s_3, \dots$ due to shared hidden layer weights.
- Bootstrapping:** We update $Q(s, a)$ toward a target that *itself depends on the network's own output* ($r + \gamma Q(s', a')$). A slight error in $Q(s', a')$ feeds directly into the target for $Q(s, a)$.
- Off-Policy Learning:** The data sampled from the replay buffer was collected by old policies from 20,000 steps ago. The data distribution does not match the distribution of the current policy, creating unstable distribution shift.

Together, these three create a vicious, self-amplifying feedback loop: an overestimation error in one state is generalized across the space, bootstrapped into future targets, and amplified exponentially until your Q-values hit 10^{15} and training collapses!

5. The Three Defensive Super-Weapons of SAC

To tame the Deadly Triad and achieve bulletproof stability on continuous robots, SAC deploys three brilliant mathematical mechanisms:

5.1 Super-Weapon 1: Experience Replay Buffer (The Photo Album)

📷 EVERYDAY ANALOGY: THE PHOTO ALBUM VS INSTANT AMNESIA

Imagine learning to play tennis, but the moment you hit a ball, you suffer complete amnesia and forget what just happened. That is on-policy learning (PPO). In contrast, an **Experience Replay Buffer** is a photographic memory album containing **1,000,000 snapshots** of every swing you ever took over the last 3 days—both aces and wild misses. Every night, you shuffle through 256 random photos from across your entire history to study your technique.

Buffer Parameter	Standard Value	Physical Function & Purpose	Tuning Danger
Capacity N	10^6 transitions	Circular FIFO buffer holding recent (s, a, r, s', d) tuples. Breaks temporal correlation across sequential video frames.	Too small (10^4): Policy forgets early lessons; Too large (10^7): GPU RAM OOM crash.
Batch Size B	256 transitions	Number of independent, uncorrelated transitions sampled uniformly at random for each stochastic gradient descent step.	Too small (32): High gradient variance; Too large (2048): Slow computation per step.
Warmup Steps	10,000 steps	Pure uniform random actions executed before any gradient updates begin. Fills the buffer with diverse boundary data.	If 0: Critic severely overfits to the initial 50 steps, locking into immediate local minima.

5.2 Super-Weapon 2: Polyak Target Networks (The Patient Teacher)

🎯 THE MOVING BULLSEYE PROBLEM

If you calculate Bellman targets using the *exact same network* you are currently updating, the target changes every single millisecond. It is like trying to shoot an arrow at a target that vibrates violently every time you pull the bowstring!

SAC maintains a separate set of **Target Critic Weights** $\bar{\theta}$. Instead of copying weights abruptly, it updates them using an exponential moving average (Polyak averaging):

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta} \quad \text{where } \tau = 0.005$$

Target Weight Half-Life ($t_{1/2}$): With $\tau = 0.005$, the target network updates by only **0.5% per gradient step**. The time required for target weights to incorporate 50% of a change in online weights is:

$$t_{1/2} = \frac{\ln(0.5)}{\ln(1 - \tau)} = \frac{-0.693}{\ln(0.995)} \approx \mathbf{138} \text{ gradient steps}$$

This sluggish target network acts as a calm, patient teacher, providing a Rock-of-Gibraltar stable target for the online critic to learn against.

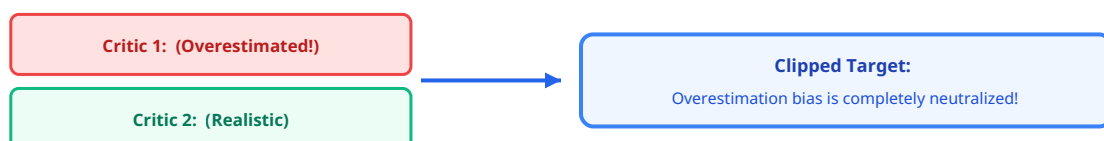
5.3 Super-Weapon 3: Clipped Twin-Q Critics (The Two Skeptical Judges)

Why do neural network critics inherently overestimate Q-values? Consider two independent noisy estimators Q_1 and Q_2 of a true value $V^* = 10.0$. Even if both estimators are zero-mean unbiased ($\mathbb{E}[Q_1] = \mathbb{E}[Q_2] = 10.0$), taking their maximum is mathematically guaranteed to be strictly greater than 10.0:

$$\mathbb{E}[\max(Q_1, Q_2)] \geq \max(\mathbb{E}[Q_1], \mathbb{E}[Q_2]) \quad (\text{Jensen's Inequality})$$

In standard continuous Q-learning, the actor intentionally searches for actions that maximize $Q(s, a)$. It will naturally find the actions where the neural network suffered an accidental positive approximation error! **The Fix:** Train **two completely independent critic networks** (Q_{ϕ_1} and Q_{ϕ_2}). When computing the Bellman target, always take the **minimum** of the two target predictions:

$$y = r(s, a) + \gamma \left(\min_{j=1,2} Q_{\phi_j}(s', a') - \alpha \log \pi(a' | s') \right)$$



6. Maximum Entropy RL: Soft Actor-Critic (SAC)

Standard RL maximizes only expected discounted reward: $\sum_t \gamma^t r_t$. In **Maximum Entropy RL**, we fundamentally change the objective: the agent is trained to maximize expected reward **PLUS the Shannon entropy of its policy!**

🧠 THE CURIOUS TOURIST IN PARIS ANALOGY

Imagine you arrive in Paris for a 2-week vacation:

- **Standard RL Agent (Reward Only):** On your first morning, you walk 20 meters from your hotel and find a corner bakery selling decent croissants (Reward = +6). Because your algorithm only maximizes reward and has zero curiosity bonus, it concludes that eating this exact croissant every single morning for 14 days guarantees a high score. You never explore Montmartre, never taste artisan sourdough, and never discover the world-class bakery 2 blocks away. You are trapped in a mediocre local optimum.

• **Maximum Entropy RL Agent (Reward + Entropy Bonus):** You are paid a monetary bonus for being **unpredictable and adventurous!** You want delicious food, but you also want high entropy (visiting diverse neighborhoods, testing different bakeries, trying macarons and crêpes). Even after finding the decent croissant, your high entropy incentive pushes you to try other streets. You quickly discover the 3-star Michelin bakery, achieving a far superior outcome!

Why this is mandatory for Robot Fruit Slicing: If a robot only maximizes reward, early in training it experiences severe negative penalties whenever its knife slips or crushes the fruit. A standard RL policy gets terrified: it learns to hold the knife frozen 2 millimeters above the fruit skin and never move, because doing nothing yields 0 penalty! The **Entropy Bonus** pays the robot to keep trying different knife angles, velocities, and sawing compliance, preventing policy freeze!

6.1 The Soft Bellman Objective & Parameter Anatomy

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \quad \text{where} \quad \mathcal{H}(\pi) = \mathbb{E}_{a \sim \pi} [-\log \pi(a | s_t)]$$

Parameter	Formal Name	Plain English Meaning	Example Value	Tuning Impact
$r(s_t, a_t)$	Extrinsic Task Reward	Physical progress points awarded by the environment (e.g. knife cutting depth minus pulp compression force).	+3.5 points	Primary task drive.
α	Entropy Temperature	The Curiosity Dial: Determines the trade-off between task exploitation and wide exploratory diversity.	$\alpha = 0.2$ (or auto-tuned)	$\alpha \rightarrow 0$: Standard RL (freezes); $\alpha \rightarrow \infty$: Pure random noise.
$\mathcal{H}(\pi(\cdot s_t))$	Shannon Entropy	A mathematical measure of how spread out and uncertain the action distribution is. A wide bell curve has high entropy; a sharp spike has low entropy.	2.4 nats	Higher entropy guarantees exploration across multiple distinct physical modes.
$\log \pi(a s_t)$	Action Log-Probability	Logarithm of the probability density of choosing action a . Because $\pi \leq 1$, $\log \pi$ is negative, so $-\log \pi \geq 0$ acts as a positive bonus!	-1.5	Larger negative log-prob = rarer action = higher exploratory bonus.

6.2 Automatic Temperature Tuning (Dual Optimization of α)

Instead of hardcoding a fixed curiosity temperature α , modern SAC treats α as a learnable parameter. We specify a **Target Entropy** $\bar{\mathcal{H}}$ (heuristically chosen as $\bar{\mathcal{H}} = -\dim(\mathcal{A})$, e.g. -14 for a 14-DoF robot). We optimize the temperature loss:

$$J(\alpha) = \mathbb{E}_{a_t \sim \pi_t} \left[-\alpha \left(\log \pi_t(a_t | s_t) + \bar{\mathcal{H}} \right) \right] \implies \nabla_{\alpha} J(\alpha) = - \left(\log \pi_t(a_t | s_t) + \bar{\mathcal{H}} \right)$$

- If policy entropy is too low ($\mathcal{H} < \bar{\mathcal{H}}$, meaning the robot is collapsing into a rigid routine too early): α automatically increases, injecting heavy exploration bonuses to force the robot to keep trying new actions.
- If policy entropy is too high ($\mathcal{H} > \bar{\mathcal{H}}$, meaning the robot already knows what to do but is still jittering randomly): α automatically decreases toward zero, letting the robot focus with laser precision on maximizing reward.

7. The Reparameterization Trick & Tanh Action Squashing

How do we backpropagate gradients through a stochastic continuous actor network?

7.1 The Backpropagation Barrier

A continuous actor outputs the parameters of a Gaussian distribution: mean $\mu_{\theta}(s)$ and standard deviation $\sigma_{\theta}(s)$. If we sample an action directly:

$$a \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s)^2)$$

The sampling operation is an **opaque random number generator**. Gradient backpropagation $\frac{\partial a}{\partial \theta}$ cannot pass through a random node! The computational graph is broken.

7.2 Kingma & Welling's Reparameterization Trick

Instead of sampling inside the computational graph, we isolate all stochasticity into an external, parameter-free noise variable $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. We compute the pre-squashed action u deterministically:

$$u = \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Now, u is a fully differentiable function of θ ! Gradients from the Critic $Q(s, a)$ flow seamlessly through u , straight through μ_{θ} and σ_{θ} , and into the Actor's neural weights:

$$\nabla_{\theta} Q(s, a) = \nabla_a Q(s, a) \cdot \nabla_u a \cdot [\nabla_{\theta} \mu_{\theta}(s) + \epsilon \odot \nabla_{\theta} \sigma_{\theta}(s)]$$

STANDARD SAMPLING (BROKEN)

✗ Gradient cannot flow through random node!

REPARAMETERIZATION TRICK (CLEAN)

✓ Seamless backpropagation from Critic to Actor!

7.3 Enforcing Physical Motor Limits: Tanh Squashing & The Jacobian Correction

A Gaussian distribution has infinite tails: an unbounded sample could demand +150.0 Nm of torque from a motor rated for ± 10.0 Nm, triggering hardware emergency shutdown. To enforce strict hardware boundaries, SAC applies a **hyperbolic tangent squashing function**:

$$a = \tanh(u) \in (-1, +1) \implies a_{\text{robot}} = a_{\text{scale}} \cdot a + a_{\text{bias}}$$

The Critical Change-of-Variables Jacobian Correction: Squashing non-linearly distorts probability densities. By the transformation theorem for continuous random variables:

$$\pi(a | s) = \mu(u | s) \cdot \left| \det \left(\frac{da}{du} \right) \right|^{-1}$$

Since $\frac{d \tanh(u)}{du} = 1 - \tanh^2(u)$, taking the logarithm yields the exact log-probability:

$$\log \pi(a | s) = \log \mu(u | s) - \sum_{i=1}^d \log(1 - \tanh^2(u_i) + \delta)$$

! THE NUMERICAL NAN TRAP: MACHINE EPSILON ($\delta = 10^{-6}$)

If $u_i \geq 10.0$, $\tanh(u_i) \rightarrow 1.0$, which causes $1 - \tanh^2(u_i) \rightarrow 0$. Evaluating $\log(0)$ produces $-\infty$, resulting in immediate **NaN gradient explosion** that irreversibly corrupts your entire network! In production code, always add a clamping safeguard: `log_prob -= torch.log(1 - a.pow(2) + 1e-6).sum(dim=-1, keepdim=True)`.

8. Concrete Step-by-Step Numerical Walkthrough: One Complete SAC Update

Let us follow the exact floating-point calculations for a single transition sampled from the replay buffer in a dual-arm slicing task:

Step	Operation	Equation / Code Formula	Calculated Value	Physical Interpretation
1	Sampled Transition	$(s, a, r, s', d) \sim \mathcal{D}$	$r = +3.50$ $d = 0$ (alive)	Knife penetrated optimal blade sheath, slicing ongoing.
2	Hyperparameters	γ, α	$\gamma = 0.99$ $\alpha = 0.20$	Standard discount factor, entropy curiosity weight.
3	Next Action Sampling	$u' = \mu(s') + \sigma(s') \odot \epsilon$ $a' = \tanh(u')$	$u' = [0.8, -0.4]$ $a' = [0.664, -0.380]$	Actor evaluates stochastic candidate continuous action.
4	Squashed Log-Prob	$\log \pi(a' s') = \log \mu(u') - \sum \log(1 - a'^2)$	$\log \pi(a' s') = -1.45$	Negative log-probability bonus: $-\alpha \log \pi = -0.29$.
5	Twin Target Critic Evaluation	$Q_{\bar{\phi}_1}(s', a'), Q_{\bar{\phi}_2}(s', a')$	$Q_1 = 34.20$ $Q_2 = 29.50$	Critic 1 overestimated, Critic 2 conservative.
6	Clipped Soft Target y	$y = r + \gamma(\min(Q_1, Q_2) - \alpha \log \pi(a' s'))$	$3.50 + 0.99(29.50 + 0.29)$ $= 3.50 + 29.49 = \mathbf{32.99}$	Pessimistic target close to ground truth label.
7	Current Q Estimates	$Q_{\phi_1}(s, a), Q_{\phi_2}(s, a)$	$Q_1 = 31.20$ $Q_2 = 30.80$	Online critic outputs for sampled (s, a) transition.
8	Critic Loss Computation	$\mathcal{L}_Q = \frac{1}{2}[(Q_1 - y)^2 + (Q_2 - y)^2]$	$\frac{1}{2}[(31.2 - 32.99)^2 + (30.8 - 32.99)^2]$ $= \frac{1}{2}[3.20 + 4.80] = \mathbf{4.00}$	Mean squared Bellman residual loss, backpropagated to update weights ϕ_1, ϕ_2 .
9	Actor Loss Computation	$\mathcal{L}_\pi = \alpha \log \pi(\tilde{a} s) - \min(Q_1(s, \tilde{a}), Q_2(s, \tilde{a}))$	$0.20(-1.10) - 31.50$ $= -0.22 - 31.50 = \mathbf{-31.72}$	Minimizing this loss simultaneously improves action and increases entropy!
10	Polyak Target Update	$\bar{\phi}_j \leftarrow 0.005\phi_j + 0.995\bar{\phi}_j$	$0.005(1.20) + 0.995(1.00)$ $= \mathbf{1.001}$	Target critic weights updated via microscopic 0.5% stability.

9. Diary of an SAC Robot Training Run (0 to 1,000,000 Steps)

What happens when you launch SAC on a dual-arm robot in NVIDIA Isaac Lab?

- **Phase 1: Warmup & Buffer Priming (Steps 0 – 10,000):** The robot executes pure uniform random motor torques. The arms flail randomly across 3D space, dropping knives and bumping into tables. No neural network weights are updated. 10,000 transitions populate the circular replay buffer.
- **Phase 2: High Entropy Exploration (Steps 10,000 – 100,000):** Training starts. α is high (≈ 0.20). Critic loss is large (≈ 15.0) as it struggles to predict value. The entropy bonus prevents premature convergence. The robot accidentally touches the fruit and registers a $+2.0$ contact reward.
- **Phase 3: Twin-Q Consolidation & Coordination (Steps 100,000 – 400,000):** Twin critics converge; overestimation bias stays near zero. The actor learns coordinated dual-arm behavior: the left arm stabilizes the fruit with the fork while the right arm aligns the blade at a 15° slicing tilt.
- **Phase 4: Low Entropy Mastery (Steps 400,000 – 1,000,000):** α automatically decays to ≈ 0.02 . The policy transitions from exploratory searching to laser precision. The robot executes smooth 80 mm sawing strokes with sub-millimeter depth accuracy (1.2 ± 0.1 mm/stroke) and minimal pulp crush ($F_z < 3.0$ N). Success rate reaches 97%.

10. Complete Production-Grade PyTorch SAC Implementation

Here is the clean, self-contained implementation of the modern Soft Actor-Critic algorithm with Twin Critics, Tanh Squashing, and Automatic Temperature Tuning:



COMPLETE PRODUCTION-GRADE SOFT ACTOR-CRITIC (PYTORCH)

```
import torch
import torch.nn as nn
from torch.distributions.normal import Normal

class ReparameterizedTanhGaussianActor(nn.Module):
    # Continuous policy network with reparameterization and Tanh squashing
    def __init__(self, state_dim, action_dim, hidden_dim=256, action_scale=1.0):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, action_dim)
        self.log_std_head = nn.Linear(hidden_dim, action_dim)
        self.action_scale = action_scale

    def forward(self, state):
        features = self.net(state)
        mean = self.mean_head(features)
        # Clamping log_std prevents numerical explosion and distribution collapse
        log_std = torch.clamp(self.log_std_head(features), min=-20, max=2)
        return mean, log_std

    def sample(self, state):
        mean, log_std = self.forward(state)
        std = log_std.exp()
        # Reparameterization Trick: u = mean + std * epsilon
        dist = Normal(mean, std)
        u = dist.rsample()
        # Hyperbolic tangent action squashing
        action = torch.tanh(u)
```

```

# Enforce change-of-variables Jacobian correction for log-probability
log_prob = dist.log_prob(u) - torch.log(1.0 - action.pow(2) + 1e-6)
log_prob = log_prob.sum(dim=-1, keepdim=True)
return action * self.action_scale, log_prob

```

```

class TwinContinuousCritic(nn.Module):
    # Two independent Q-networks trained in parallel to defeat overestimation
    def __init__(self, state_dim, action_dim, hidden_dim=256):
        super().__init__()
        # Critic 1
        self.q1 = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )
        # Critic 2
        self.q2 = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, state, action):
        sa = torch.cat([state, action], dim=-1)
        return self.q1(sa), self.q2(sa)

class SACAgent:
    # Full Soft Actor-Critic agent with Twin-Q and Auto-Tuned Entropy
    def __init__(self, state_dim, action_dim, lr=3e-4, gamma=0.99, tau=0.005):
        self.gamma = gamma
        self.tau = tau
        self.actor = ReparameterizedTanhGaussianActor(state_dim, action_dim)
        self.critic = TwinContinuousCritic(state_dim, action_dim)
        self.critic_target = TwinContinuousCritic(state_dim, action_dim)
        self.critic_target.load_state_dict(self.critic.state_dict())

        self.actor_opt = torch.optim.Adam(self.actor.parameters(), lr=lr)
        self.critic_opt = torch.optim.Adam(self.critic.parameters(), lr=lr)

        # Automatic Entropy Temperature Tuning
        self.target_entropy = -float(action_dim)
        self.log_alpha = torch.zeros(1, requires_grad=True)
        self.alpha_opt = torch.optim.Adam([self.log_alpha], lr=lr)

    @property
    def alpha(self):
        return self.log_alpha.exp().item()

    def update(self, batch):
        states, actions, rewards, next_states, dones = batch

        # 1. CRITIC UPDATE: Clipped Twin-Q Bellman Target
        with torch.no_grad():
            next_actions, next_log_probs = self.actor.sample(next_states)
            q1_target, q2_target = self.critic_target(next_states, next_actions)

```

```

min_q_target = torch.min(q1_target, q2_target) - self.alpha * next_log_probs
y = rewards + (1.0 - dones) * self.gamma * min_q_target

q1_current, q2_current = self.critic(states, actions)
critic_loss = 0.5 * (nn.functional.mse_loss(q1_current, y) +
                    nn.functional.mse_loss(q2_current, y))

self.critic_opt.zero_grad()
critic_loss.backward()
self.critic_opt.step()

# 2. ACTOR UPDATE: Maximize Expected Q + Entropy
new_actions, log_probs = self.actor.sample(states)
q1_new, q2_new = self.critic(states, new_actions)
min_q_new = torch.min(q1_new, q2_new)
actor_loss = (self.alpha * log_probs - min_q_new).mean()

self.actor_opt.zero_grad()
actor_loss.backward()
self.actor_opt.step()

# 3. ENTROPY TEMPERATURE UPDATE
alpha_loss = -(self.log_alpha * (log_probs + self.target_entropy).detach()).mean()
self.alpha_opt.zero_grad()
alpha_loss.backward()
self.alpha_opt.step()

# 4. POLYAK TARGET UPDATE
for param, target_param in zip(self.critic.parameters(), self.critic_target.parameters()):
    target_param.data.copy_(self.tau * param.data + (1.0 - self.tau) * target_param.data)

```

11. Practitioner's Field Guide: 6 Continuous RL Failure Modes & Fixes

When training continuous RL on physical or simulated robots, these 6 insidious bugs account for 95% of all failed projects:

Failure Symptom	The Hidden Root Cause	The Production-Proven Fix
1. Q-Values Explode to $+10^8$	Polyak target parameter τ set too aggressively ($\tau = 0.05$ instead of 0.005) or Twin-Critic minimum was accidentally replaced with an average. The Deadly Triad triggers runaway positive feedback.	Set $\tau \in [0.005, 0.01]$. Verify that target evaluation strictly takes <code>torch.min(q1, q2)</code> rather than <code>0.5*(q1 + q2)</code> .
2. Policy Hits ± 1.0 (Saturated Torques)	Entropy temperature α decayed to zero prematurely, or rewards are scaled too high (e.g. $r = +500$). The network saturates motor limits in a desperate effort to maximize reward.	Normalize environment rewards by multiplying by 0.1 or 0.01. Ensure automatic entropy tuning has target entropy $\bar{\mathcal{H}} = -\dim(\mathcal{A})$.
3. Training NaNs at Step 45,000	Jacobian correction evaluates $\log(1 - a^2)$ where $a = \pm 1.00000$, resulting in $\log(0) = -\infty$. Gradients multiply into NaNs, wiping out all neural weights.	Always clamp inside log: <code>torch.log(1.0 - a.pow(2) + 1e-6)</code> . Also clamp policy <code>log_std</code> between <code>[-20, +2]</code> .
4. Robot Freezes / Does Nothing	Entropy bonus is too small or absent ($\alpha = 0$). Robot incurs small negative penalties for moving, realizes that standing completely still yields 0 penalty, and freezes forever.	Initialize $\alpha = 0.2$ or higher. Provide small shaping rewards for progress towards the target to overcome initial inertia.

Failure Symptom	The Hidden Root Cause	The Production-Proven Fix
5. Catastrophic Replay RAM OOM	Storing high-dimensional float64 states or raw RGB camera images directly in Python lists causes CPU/GPU memory to leak and Linux OOM killer to terminate the script.	Pre-allocate monolithic continuous <code>torch.float32</code> NumPy ring buffers. For vision, store compressed JPEG bytes or low-dimensional latent vectors.
6. Cold-Start Policy Collapse	Updating neural network parameters on step 1 when the replay buffer contains only 5 transitions. The critic severely overfits to 5 points and crashes.	Always implement a strict warmup phase : execute 10,000 random exploration steps before calling <code>agent.update()</code> for the first time.

12. Continuous RL Algorithm Selection Cheat Sheet

Feature / Property	DDPG (2015)	TD3 (2018)	SAC (2018)	PPO (Lecture 10)
Policy Type	Deterministic	Deterministic	Stochastic (MaxEnt)	Stochastic
Sample Efficiency	Moderate	High	Very High (Best in Class)	Moderate (On-Policy)
Exploration Mechanism	Additive Gaussian noise	Target policy smoothing	Intrinsic Entropy Bonus $\alpha \mathcal{H}$	Policy entropy penalty
Overestimation Protection	None (Severe bias)	Clipped Double-Q	Clipped Double-Q	GAE Advantage Clipping
Hyperparameter Tuning	Extremely brittle	Moderate	Very Robust (Auto α)	Very Robust
Ideal Robotics Application	Deprecated / Historical	Simple low-DoF joints	Complex manipulation, soft contacts, high DoF	Massively parallel sim (Isaac Lab locomotion)

CS285 Lecture 8 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An